Anyaugo Space-Tech Ltd.

SECURITY TRAINING | LAB EVIDENCE



PROJECT 10 / UNIFIED EVIDENCE PORTFOLIO

Cloudflare Edge Security

Put a vulnerable web app behind a free CDN/WAF edge; attack it, block it, watch the block, and recover the security configuration after it is wiped — entirely in a controlled lab.

Prepared by: Anyaugo Space-Tech Ltd. — Security Training track
Environment: OWASP Juice Shop (Docker) behind Cloudflare's free edge on the owned domain **anyaugo-drs.com**, connected by a Cloudflare Tunnel; attacker on a Kali VM
Evidence: Original screenshots across the deploy → attack → defend → monitor → recover stages, plus prepared records for scope, architecture and responsible use
Frameworks: OWASP Top 10 · Cloudflare Scans & Penetration Testing policy · NIST SP 800-115 · NIST CSF 2.0
Report date: 11 October 2026

BUSINESS QUESTION

A company exposes a web app to the internet. **Can a free CDN/WAF edge stop real attacks before they reach the server, hide the origin so it cannot be bypassed, and can the security configuration be recovered after a destructive change — without paying a ransom?**

VERIFIED RESULT

Within the free tier's limits, yes. A SQL-injection the bare origin processes is **blocked at the edge (403) and logged**; a detector alerts on it. The origin has **no public IP** (Cloudflare Tunnel), so the WAF cannot be bypassed. Deleting the rule lets the same request reach the origin and trip a database error (500); restoring it from an off-platform backup returns the request to 403 — **no ransom**. One result is stated as plainly as the wins: the app's **DOM-based XSS cannot be blocked by any WAF** and needs a code-level fix.

PORTFOLIO BOUNDARY

A controlled edge-security exercise on a Cloudflare zone, domain and origin the agency owns, using a deliberately-vulnerable disposable target and synthetic data, under Cloudflare's policy permitting testing of your own assets. No client systems, no real data.

INTRODUCTION

Defending a web app at the edge

What this portfolio demonstrates

The question behind the work

Almost every public web app now sits behind a CDN/WAF edge — Cloudflare, Akamai, AWS CloudFront. The edge *can* stop attacks before they reach the server, but only if it is configured to, and only for the attacks it can actually see. The honest question is not whether a WAF is "on", but **what it blocks, what it cannot, whether the server can be reached around it, and whether the protective configuration can be recovered** after someone deletes it. That is this project.

Deploy, attack, defend, monitor, recover

The work runs the full loop a company pays an edge/security engineer for: **deploy** the app behind the edge with the origin hidden; **attack** the bare origin to show it is vulnerable; **defend** by deploying a WAF rule and proving the same attack is now blocked; **monitor** the blocks with a detector that raises alerts; and **recover** the security configuration from an off-platform backup after it is destroyed.

The target, and what counts as a result

The target is **OWASP Juice Shop** — a deliberately-vulnerable app — run in a disposable Docker container the agency owns. Because the app is *meant* to be vulnerable, the point is not to find its bugs; it is to prove the **edge defence, monitoring and recovery around it**, and to be precise about the one class of flaw an edge genuinely cannot fix. Every record is synthetic (`admin@juice-sh.op`); no real customers, credentials or production data are used.

Honesty, stated up front

The free tier blocks query-string attacks but not request bodies; the detector is a purpose-built poller, not a full SIEM; and the recovery loop proves an off-platform backup, not yet a true immutable vault. Each of these is carried openly into the findings rather than smoothed over.

HOW TO READ THIS REPORT

Evidence index

Each stage, what it establishes, and how many captures back it

TEST	STAGE	CAPTURES	WHAT IT ESTABLISHES
01	Scope, Authorization & Architecture	Records	Rules of engagement, Cloudflare's own-asset testing policy, and the secure-edge design — fixed before the build
02	Target & Secure Origin	12	Juice Shop served through a Cloudflare Tunnel (no public origin IP); edge hardened with visitor TLS (Always-HTTPS, HSTS, min TLS 1.2)
03	Baseline Attack (Exposed)	3	Against the bare origin, a SQL-injection login bypass succeeds and a DOM-based XSS payload is injected client-side
04	Edge Defense Deployed	1	One fully-understood custom WAF rule — url-decode + lowercase to defeat evasion
05	Attack Blocked & Tuned	4	The same SQLi is now 403 at the edge, normal traffic unaffected, and the block is logged
06	Monitoring & Detection	1	A detector polls Cloudflare's firewall-events API and alerts on blocks, source identity self-masked
07	Configuration Backup & Recovery	11	WAF rule and DNS backed up, destroyed, and restored — the recover-without-ransom loop, twice
09	Privacy & Responsible Use	Record	Responsible handling of evidence and AI-assisted work

Test 08 is this report. Headline captures are reproduced one per stage in the pages that follow; all original screenshots are preserved, unretouched (except where sensitive source data is redacted), in the project's per-test folders under their descriptive filenames. Captures are named by letter within each stage (Test-2A, Test-2B…). Every finding stage also carries a plain-language Test-NN-finding-notes.md record in its folder, and the WAF rule and detector are preserved as a JSON backup and a Python script.

TEST 01 / SECURITY TRAINING | LAB EVIDENCE

Scope and Authorization Record

Prepared record · before the build

Purpose

Project 10 is an authorized edge-security exercise — deploy a web app behind Cloudflare, attack it, defend it, monitor the defence, and recover the configuration — carried out on a Cloudflare zone, domain and origin the agency owns. This record fixes the targets, activities and exclusions before any deployment or test.

Lab ownership and scope

The Cloudflare zone **anyaugo-drs.com**, the subdomain **juice.anyaugo-drs.com**, the Cloudflare Tunnel that connects it, and the origin — an OWASP Juice Shop container running in Docker on the agency's own host — are all owned and controlled by the agency. The attacker is the agency's own Kali VM. No other zone, no Cloudflare infrastructure, and no third-party system is in scope.

Cloudflare's own-asset testing policy

Cloudflare's published Scans and Penetration Testing policy permits a customer to test their own assets — their own IPs, Cloudflare-designated IPs, and their own registered DNS — and never `*.cloudflare.com` (bug-bounty only). Scans must be throttled and non-destructive; penetration tests are expected to run against the fuller managed rulesets. **No denial-of-service or request flooding** is performed.

Approved activities

Deploy the origin behind the Tunnel with no public IP; configure visitor TLS (Always Use HTTPS, HSTS, minimum TLS 1.2); attack the bare origin and the edge from a customer's position (SQL-injection, cross-site scripting); deploy and tune a WAF custom rule; verify the block at the edge and in Cloudflare's log; build a detector against the firewall-events API; and back up, destroy and restore the security configuration to prove recovery.

Exclusions and stop conditions

No testing of Cloudflare's own infrastructure or any asset not owned. No DoS/DDoS or request flooding. No real personal, financial or production data — all records are synthetic. Testing stops and the event is recorded if any real data or credential appears, or if any activity would reach beyond the owned zone and origin.

Authorization note

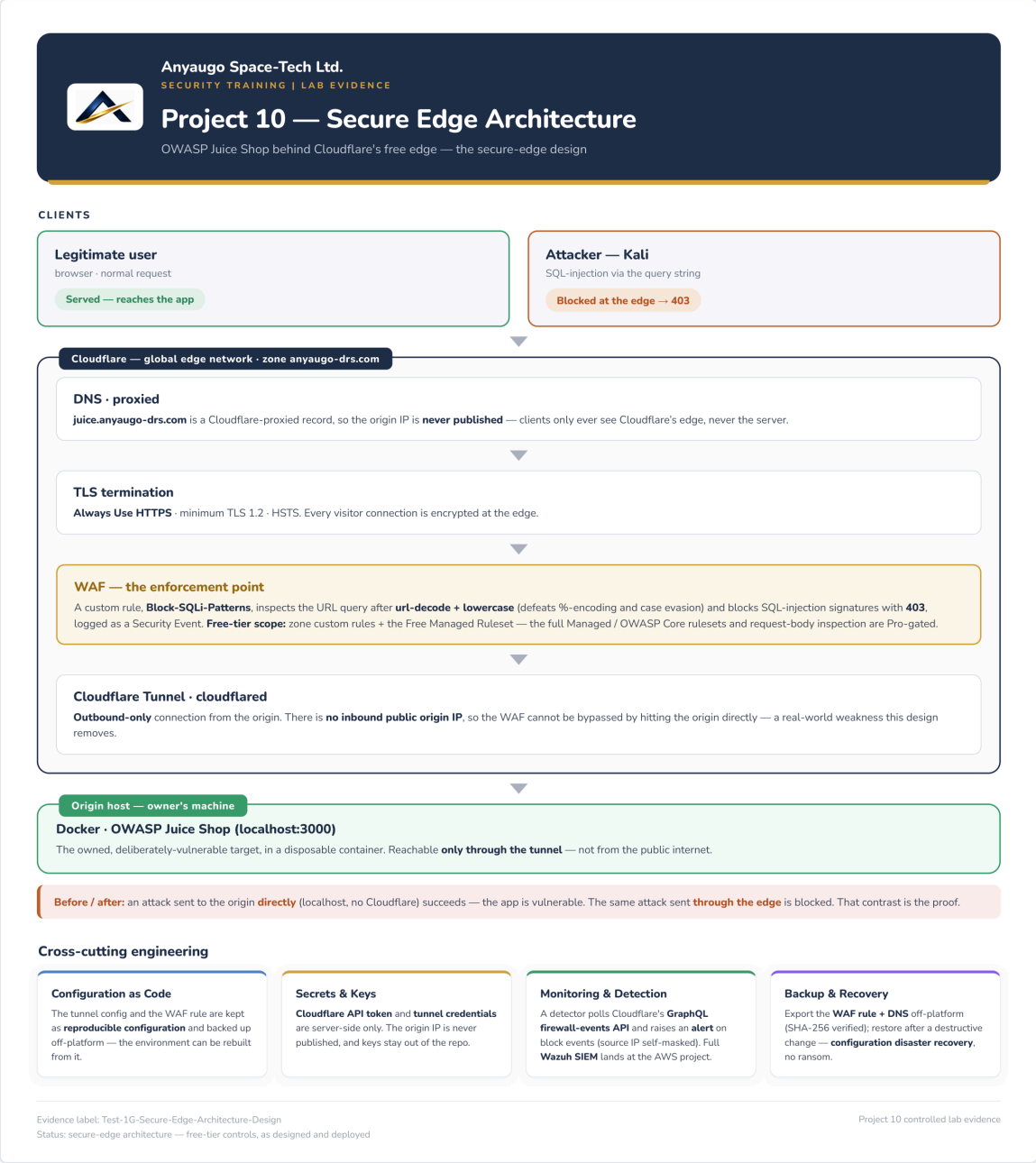
This record does not claim client or company authorization; it documents that the work stays on infrastructure the agency owns and controls, within Cloudflare's own-asset policy and the Rules-of-Engagement principle of NIST SP 800-115. It is a scope record for a training lab, not a legal opinion.

ARCHITECTURE / EDGE-ARCHITECT STAGE

The secure-edge design, before the build

Prepared record · the design that precedes the build

Before any resource was created, the system was designed the way an edge architect sets the shape of a build: who may call it, the path a request takes, and the single point where attacks are actually enforced against. Every later stage implements this diagram, and it is drawn to match what the free tier can genuinely deliver.



Evidence note: Clients reach **DNS** → **TLS** → the **WAF enforcement point** → the **Cloudflare Tunnel** → the Docker origin, which has **no public IP**. The diagram is deliberately scoped to the free-tier controls actually deployed — the Pro-gated rulesets and a full SIEM are named as where this scales, not claimed here.

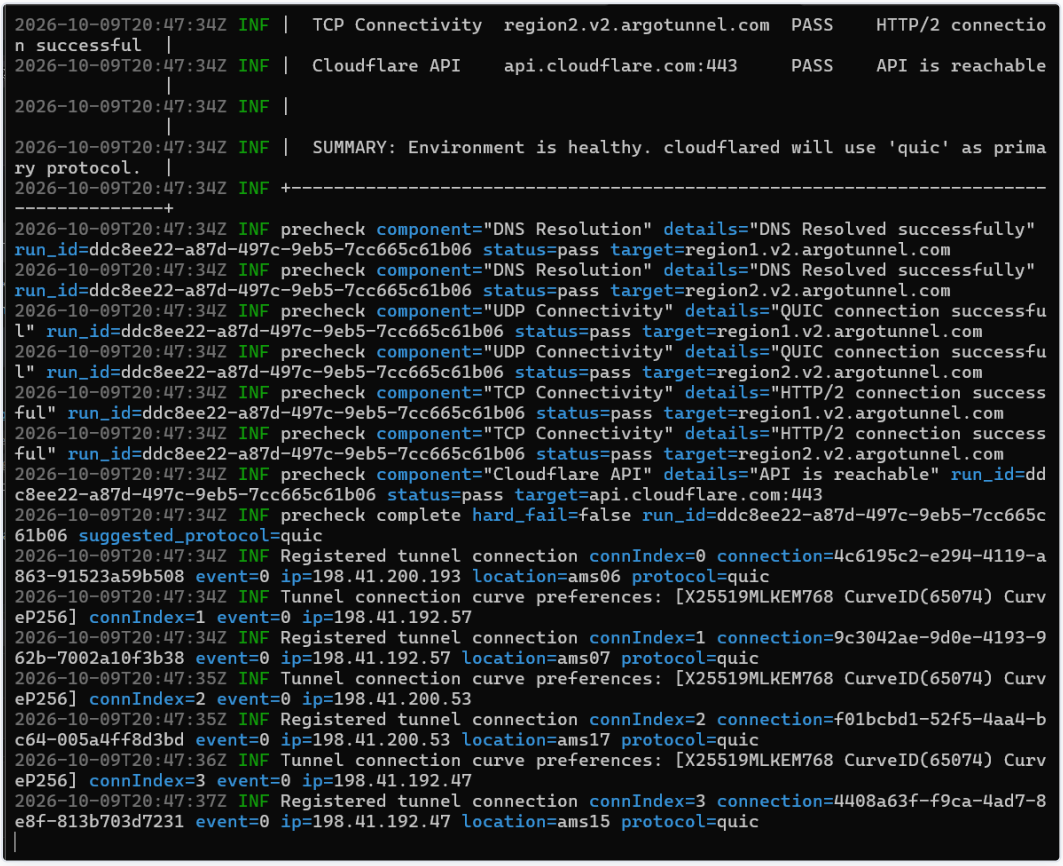
Test-1G-secure-edge-architecture-design.png

TEST 02 · 2H

The origin, hidden behind a tunnel

Original lab screenshot · Cloudflare Tunnel

The origin is connected to Cloudflare by an outbound-only **Cloudflare Tunnel** (`cloudflared`), not by opening a port. The tunnel registers four healthy connections to Cloudflare's edge, and traffic for `juice.anyaugo-drs.com` flows in over it — so the origin has **no inbound public IP at all**.



Evidence note: This is the single most important design choice: because the origin is never addressable from the internet, an attacker **cannot bypass the WAF by hitting the server directly** — the classic real-world WAF-bypass is removed by architecture, not by a rule. The install, login, tunnel-create, DNS-route and config-validate captures (2A–2G) are preserved in the Test-02 folder.

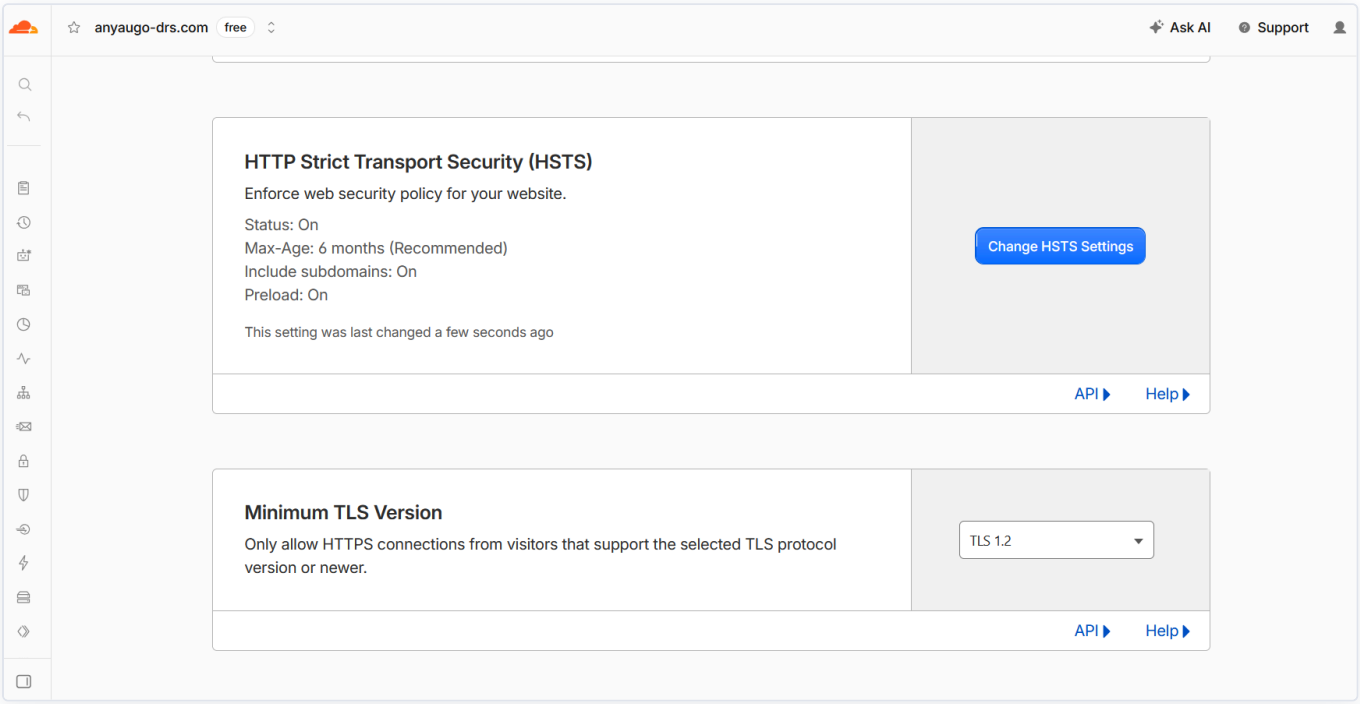
Test-2H-2-tunnel-connected-registered.png

TEST 02 · 2I

Edge hardening: visitor TLS

Original lab screenshot · transport security

With the origin hidden, the edge itself is hardened so every visitor connection is encrypted and modern. **Always Use HTTPS** is on (plain-HTTP requests are redirected), **HSTS** is on (6 months, include-subdomains, preload), and the **minimum TLS version** is set to **1.2** — older, weaker protocols are refused outright.



Evidence note: Every visitor connection is forced onto modern TLS (1.2+), and HSTS with preload means browsers refuse to connect over plain HTTP at all — closing off downgrade and SSL-stripping attacks at the edge. The Always-Use-HTTPS capture (2I-1) is preserved alongside this one in the Test-02 folder.

Test-2I-2-hsts-and-min-tls12.png

TEST 03 · 3B

The bare origin is vulnerable

Original lab screenshot · baseline attack

Before any WAF, the origin is attacked from Kali to establish the baseline. A classic SQL-injection login bypass — `admin@juice-sh.op'--` in the login request — succeeds: the app returns **HTTP 200** and issues a valid admin **JWT**. A DOM-based cross-site-scripting payload is also injected into the search page (next finding).

```
┌──(project10-kali㉿project10-kali)-[~]
└─$ curl -s -X POST http://192.168.1.116:3000/rest/user/login \
  -H "Content-Type: application/json" \
  --data '{"email":"admin@juice-sh.op'"'"'--","password":"x"}' \
  | python3 -c "import sys,json; a=json.load(sys.stdin)['authentication']; t=a['token']; print('R
ESULT: HTTP 200 — LOGGED IN AS', a['umail']); print('ADMIN JWT ISSUED:', t[:4] + '*'*44, f'(redac
ted · {len(t)} chars)')"

RESULT: HTTP 200 — LOGGED IN AS admin@juice-sh.op
ADMIN JWT ISSUED: eyJ0********************************************* (redacted · 717 chars)

┌──(project10-kali㉿project10-kali)-[~]
└─$
```

Evidence note: The injection is a genuine authentication bypass against the unprotected origin — the token is captured as proof. **Responsible handling:** the JWT is **asterisk-masked in the command itself** (`eyJ0***… redacted · 717 chars`) — the `eyJ` prefix proves it is a real token while the value never appears. The DOM-XSS capture (3C) is in the Test-03 folder.

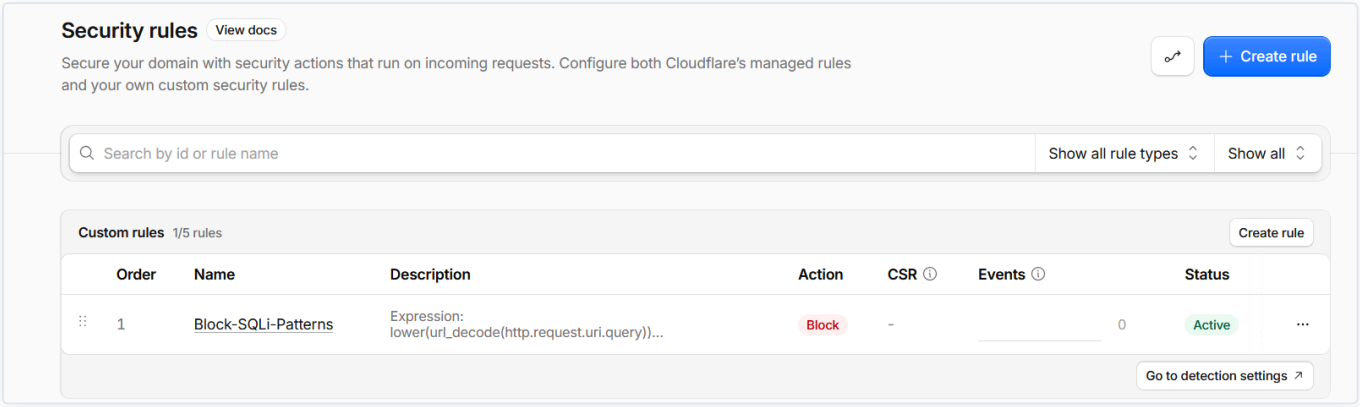
Test-3B-sqli-login-bypass-admin-token.png

TEST 04 · 4

One rule, fully understood

Original lab screenshot · WAF custom rule

A single custom WAF rule, **Block-SQLi-Patterns** (action: Block), is deployed. It inspects the URL query after **url-decode** (defeating %-encoding evasion) and **lowercase** (defeating case evasion), and blocks requests containing common SQL-injection signatures — `union select`, `' or 1=1`, `'--`, `sleep(`.



Evidence note: One rule the operator fully understands beats a wall of opaque managed rules you cannot explain. **Free-tier reality:** the full Cloudflare Managed / OWASP Core rulesets, WAF-attack-score and request-body inspection are Pro-gated; this rule covers query-string SQLi, which is what the free tier can see. The exact expression is preserved as a JSON backup in the Test-07 folder.

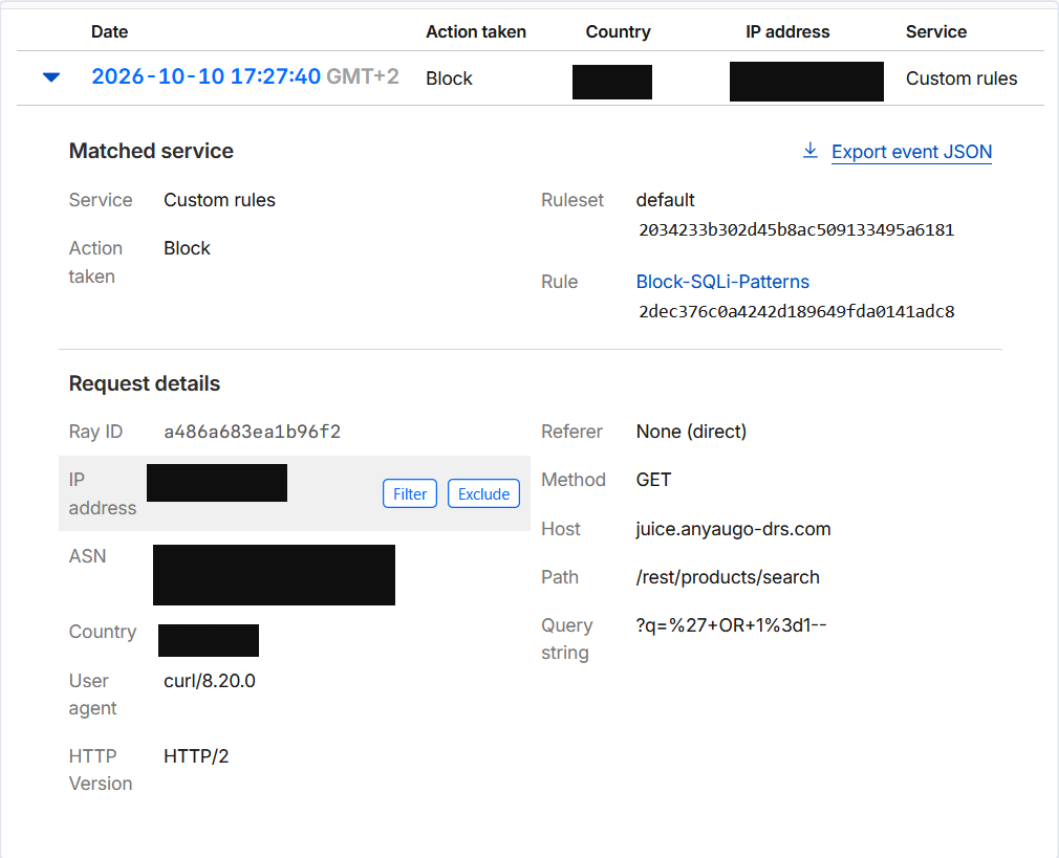
Test-4-custom-rule-sqli-block.png

TEST 05 · 5D

Blocked at the edge — and logged

Original lab screenshot · Cloudflare Security Event

The before/after is run from Kali: a normal search via the edge returns **200** (5A); the SQL-injection `?q=' OR 1=1--` via the edge is **403 — blocked** (5B); the same payload sent to the direct origin is *not* blocked, confirming the app itself is still vulnerable (5C). Cloudflare's own Security Event, shown here, records the block.



Evidence note: The log shows exactly what Cloudflare stopped: Action **Block**, Rule **Block-SQLi-Patterns**, Host `juice.anyaugo-drs.com`, Path `/rest/products/search`, the SQLi query, and the `curl` user-agent — while the normal search was untouched. The attacker's source IP, ASN and geo are redacted (operator's own connection); the payload, rule and request path are the evidence and are kept visible.

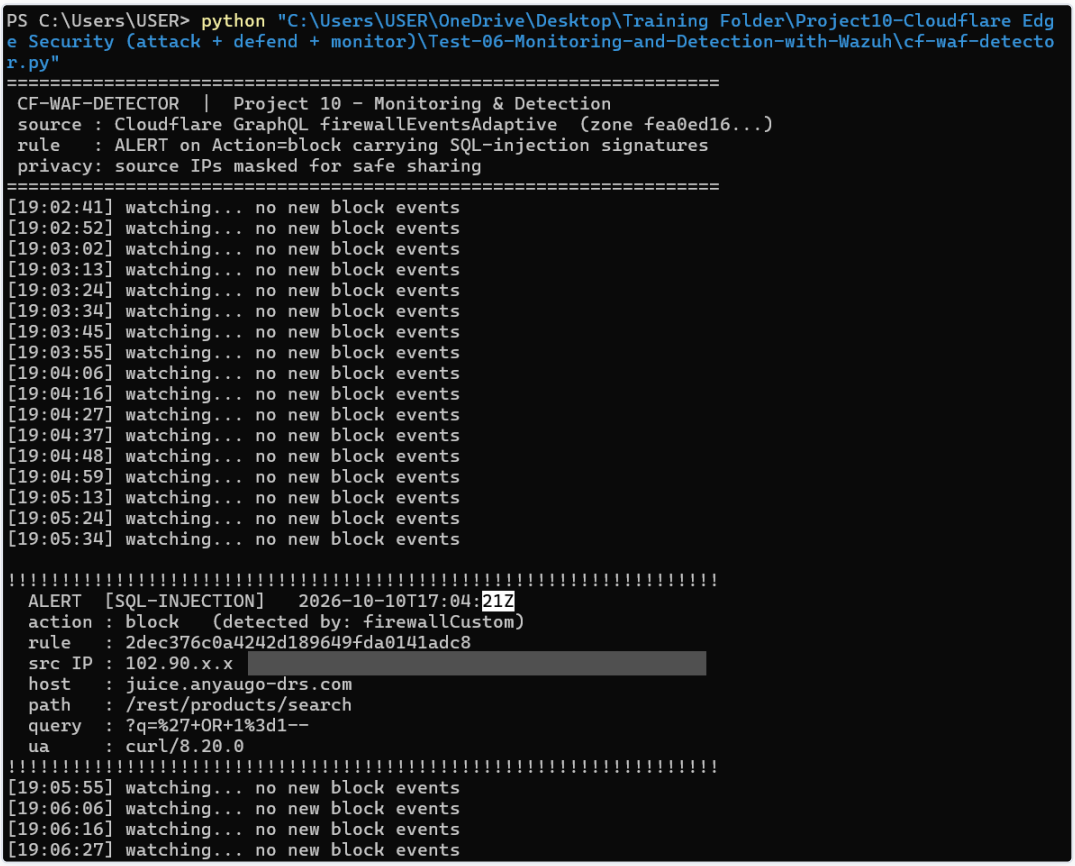
Test-5D-cloudflare-security-event-blocked.png

TEST 06 · 6

Detection: from block to alert

Original lab screenshot · the SOC half

Blocking is only half the job; the block must be *seen*. A lightweight detector (`cf-waf-detector.py`, standard-library Python) polls Cloudflare's GraphQL **firewallEventsAdaptive** API — the free tier has no Logpush, so events are pulled on an interval — applies a detection rule, and raises an **ALERT** on block events carrying SQL-injection signatures.



Evidence note: This is the blue-team loop — **collect → detect → alert** — against a real provider API, with the detector running on the host (SOC) while the attack comes from Kali (role separation). The alert's source IP is auto-masked (`102.90.x.x`) and its ASN/geo redacted for safe sharing. The full classical Wazuh SIEM is built where real compute exists — the AWS project.

Test-6-detector-sqli-alert.png

TEST 07 · 7F

Recover the config — no ransom

Original lab screenshot · recovery verified

Cloudflare's edge holds no customer data, so the asset that must survive a destructive event is the **configuration**. The loop is proven: back up the WAF rule (JSON) and DNS off-platform → verify integrity (SHA-256) → delete the rule → confirm the breach (the SQLi now reaches the origin and trips a 500) → restore the rule from the backup → verify the same request is **403 again**, shown here.

```
┌──(project10-kali㉿project10-kali)-[~]
└─$ curl -s -o /dev/null -w "after restore -> HTTP %{http_code}\n" -G "https://juice.anyaugo-drs.com/rest/products/search" --data-urlencode "q=' OR 1=1--"

after restore -> HTTP 403

┌──(project10-kali㉿project10-kali)-[~]
└─$
```

Evidence note: The request flips **403 → 500 → 403** as the rule is removed and restored — the control is recovered from an off-platform backup, no ransom. A second loop proves the same at the **DNS layer** (7G delete → site unreachable; 7H restore; 7I HTTP/2 200 again). **Calibrated finding:** the 500 is an error-based SQLi indicator from the origin, not a clean data-extraction — detailed on page 16.

Test-7F-sqli-blocked-after-restore.png

FINDINGS

What the engagement proved — and its honest limits

One finding against the target, two stated limits

#	FINDING / LIMIT	SEV.	WHAT IT MEANS	STATUS
P10-F1	DOM-based XSS cannot be blocked by any WAF	**Medium**	The search-XSS payload lives in the URL fragment the browser never sends to the server, so no edge WAF ever sees it; the fix is code-level output encoding	**Documented**
P10-L1	Free-tier WAF inspects the query string, not bodies	**Info**	Body-based injection (e.g. the Test-03 login bypass) needs Pro request-body inspection; out of scope on free	**Residual**
P10-L2	Error-based result is an indicator, not a clean exploit	**Info**	With the rule removed, the SQLi returns a 500 SQL error from the origin — proof input reached the DB, not a 200+data extraction	**Calibrated**

Controls that held

The edge blocks server-visible attacks. The SQL-injection that the bare origin processes returns **403** through Cloudflare, while a normal search returns **200** — proven as a clean before/after.

The origin cannot be bypassed. The Cloudflare Tunnel leaves the origin with **no public IP**, so an attacker cannot route around the WAF by hitting the server directly.

Blocks are detected and the config is recoverable. Each block becomes an alert, and the WAF rule and DNS were restored from an integrity-verified off-platform backup after deletion — **no ransom**.

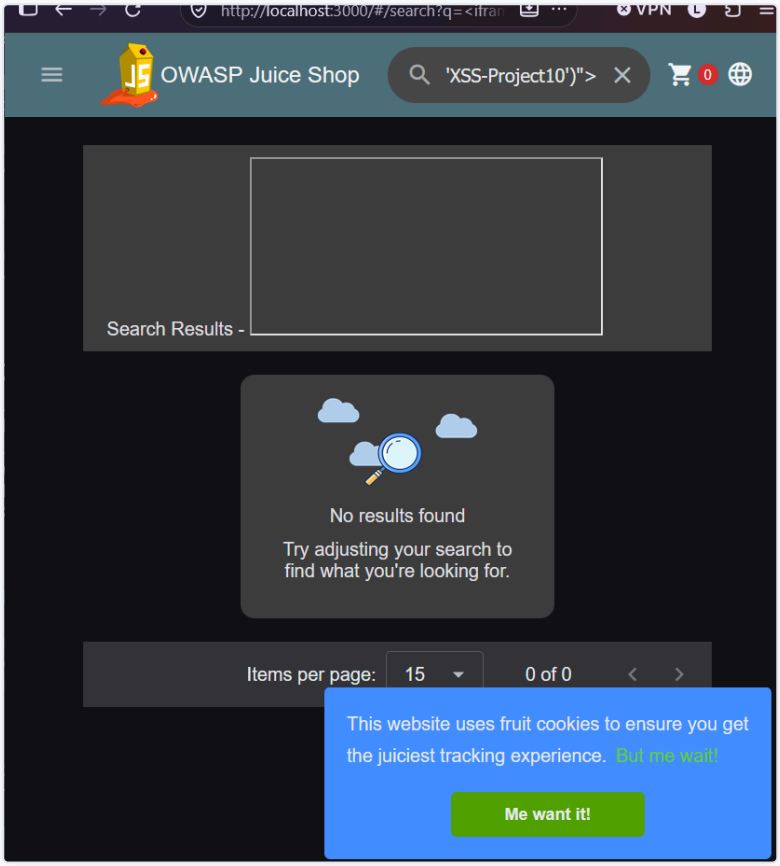
Because the target is deliberately vulnerable, the value is not a count of its bugs but the proof of the edge defence, monitoring and recovery around it — and the precision about the one flaw an edge cannot fix.

FINDING P10-F1

The DOM XSS a WAF cannot block

Recorded severity: Medium · the engagement's key lesson

This is the finding that separates someone who has *used* a WAF from someone who *understands* one. Juice Shop's search reflects an attacker's markup straight into the page — a client-side (DOM) cross-site-scripting injection — yet it is impossible to block at the edge, and the reason is architectural, not a tuning gap.



Observed: the raw `<iframe>` payload is reflected into the page and rendered client-side — the injected frame appears in the search results (shown), and the address bar shows the payload sitting in the URL **fragment**: `#/search?q=<iframe…>`. Everything after the `#` is the fragment, and browsers **never send it to the server**, so it stays entirely client-side.

Why the WAF is powerless. A WAF inspects requests that reach the server. If the malicious value never leaves the browser, Cloudflare never sees it — there is nothing to match on. No rule, on any plan, can block a purely client-side (DOM) vulnerability.

The correct fix. Output encoding and a strict Content-Security-Policy in the application code — not an edge rule. A WAF protects the *server* edge; client-side execution is the application's own responsibility. Knowing which control fixes which flaw is the point of the finding.

Test-3C-dom-xss-payload-injected.png

FRAMEWORK MAPPING

OWASP Top 10 & NIST CSF 2.0

How an edge-security engagement is reported

REFERENCE	AREA	EVIDENCE IN THIS PROJECT	RESULT
A03:2021	Injection (SQLi)	SQL-injection blocked at the edge (403) and logged; bare origin still vulnerable (Tests 03, 05)	**Edge-mitigated**
A03:2021	Injection (XSS)	DOM injection reflected client-side; not WAF-blockable, needs code-level encoding — P10-F1 (Tests 03, F1)	**Documented**
A05:2021	Security Misconfiguration	Origin hidden by tunnel (no public IP); WAF enforced; secure edge defaults (Tests 02, 04)	**In place**
A02:2021	Cryptographic Failures	Always-HTTPS, HSTS (preload), minimum TLS 1.2 at the edge (Test 02)	**In place**
NIST CSF 2.0	Detect (DE.CM)	GraphQL firewall-events detector raises alerts on blocks; Security Events log (Tests 05, 06)	**Proven**
NIST CSF 2.0	Recover (RC.RP)	WAF rule + DNS restored from off-platform integrity-verified backup (Test 07)	**Proven**

Reading the mapping

The web OWASP Top 10 is the right lens because the target is a web app. Both SQL-injection and cross-site-scripting fall under **A03:2021 Injection** — and the two are deliberately split here, because the edge *mitigates* the server-visible SQLi but can only *document* the client-side XSS. Hiding the origin and enforcing the WAF are **A05 Security Misconfiguration**; the TLS hardening is **A02 Cryptographic Failures**.

Detection and recovery are mapped to the **NIST Cybersecurity Framework**: the detector and Security Events cover **Detect**, and the configuration backup-and-restore loop covers **Recover** — where ransomware resilience lives. The depth of that recovery, from an off-platform backup here to a true immutable vault later, is what scales across this cloud series.

Frameworks are reporting language, not a certification; current OWASP Top 10 (2021) and NIST CSF 2.0 versions were referenced at build time.

INTERPRETATION

What the edge does, and does not, do

Reading the results precisely

What the edge genuinely stopped

A SQL-injection that the bare origin processes was **blocked at the edge with a 403**, the normal search was untouched, and Cloudflare's own log recorded the block with the matching rule and payload. The origin's hidden address means that block **cannot be bypassed** by hitting the server directly. These are real, verified wins.

The calibrated breach finding (P10-L2)

When the rule was deleted, the same injection reached the origin and returned a **500 with** `SQLITE_ERROR: incomplete input`. That is an **error-based SQL-injection indicator** — proof that unsanitised input reached the database and broke the query. It is **not** a demonstrated data-extraction or authentication bypass, which would return **200 with unauthorised data**. The 500 came from the origin (Express/SQLite), not from Cloudflare. Stating this precisely is the difference between a measured finding and an overclaim.

What an edge cannot fix (P10-F1)

The DOM-based XSS lives in the URL fragment the browser never sends, so no WAF can see or block it. The fix is **code-level output encoding and a Content-Security-Policy**, not an edge rule. An edge protects the server perimeter; client-side execution is the application's responsibility.

Free-tier scope (P10-L1)

The free tier gives zone custom rules, one rate-limit rule, and the Free Managed Ruleset. The full Managed / OWASP Core rulesets, WAF-attack-score and **request-body inspection** are Pro-gated — so a SQLi carried in a JSON body (like the Test-03 login bypass) would need Pro to be blocked at the edge. This is documented, not worked around.

RESILIENCE

Recovering the configuration without paying

Recover-without-ransom, adapted to a stateless edge

Why configuration, not data

Cloudflare's edge is a stateless data-plane: it holds no customer data (the origin does, and here that origin is a disposable container). So the asset that must survive a destructive event is the **security configuration** — the WAF rule that enforces protection, and the DNS record that routes the service. Those are what were backed up, destroyed, and restored.

Two recovery loops, both proven

WAF rule: backed up as JSON, SHA-256 verified, deleted (the SQLi then reached the origin and tripped a 500), and restored from the backup — the request returned to 403. **DNS record:** deleted (the site became unreachable — `could not resolve host`), then restored — the site returned **HTTP/2 200 through Cloudflare** with HSTS still enforced. In both, recovery came from the agency's own backup, with no ransom.

The honest boundary

This proves the **recovery loop and an off-platform, integrity-verified backup** — the 3-2-1 principle. It is **not** yet a true immutable/air-gapped vault: the backup lives on the same machine, so an attacker who genuinely owned the host could reach it too.

Where it scales

True Write-Once-Read-Many immutability — AWS **S3 Object Lock** in a separate account whose credentials the attacker never holds — is built in the AWS project later in this series. This stage is the foundation that builds on: a recovery loop you can actually run, before the vault that makes it unbreakable.

PROPOSED PRACTICE

How this would run for a client

Proposed workflow — not completed client work

1. Scope and authorize. Confirm ownership of the zone, domain and origin, and the provider's own-asset testing policy (as Cloudflare's permits), before any attack — and keep permission to test separate from permission to move evidence to any outside service.

2. Hide the origin first. Put the origin behind an outbound-only tunnel so it has no public IP. A WAF is worth little if the server can be reached around it; removing that bypass is an architecture decision, not a rule.

3. Harden the edge. Enforce HTTPS, HSTS and a modern minimum TLS version — the transport layer — before layering on the WAF.

4. Deploy rules you understand. Start with a small number of fully-understood custom rules (url-decoding and lowercasing to defeat evasion), prove each with a clean before/after, and name honestly what the plan tier cannot inspect.

5. Monitor and alert. Pull or stream firewall events into detection that raises alerts, with source data masked for safe sharing — blocking without visibility is half a control.

6. Back up, break, and recover. Export the protective configuration off-platform, verify its integrity, rehearse deleting and restoring it, and state the residual honestly — the resilience an owner cares about most.

On running this at scale. This is the proposed shape of an edge-security engagement, offered as practice, not a record of client work performed. Across this cloud series the same deploy-attack-defend-monitor-recover loop is repeated on each major platform, which is what lets one engineer credibly run secure builds, testing, monitoring and recovery for several firms on a retainer.

ASSURANCE

What this does and does not establish

The limits of the evidence

A controlled training lab, not a client assessment

No client or company authorization is claimed. The Cloudflare zone, domain and origin were created and owned by the agency; all data is synthetic; testing stays within Cloudflare's own-asset policy. This is a portfolio exercise, not a legal opinion.

Edge defence proven, within free-tier scope

The WAF block, the hidden origin, the TLS hardening, the detection loop and the recovery loop were all demonstrated. But the edge is not claimed to be a complete WAF: the free tier inspects the **query string, not request bodies**, and the full managed/OWASP rulesets are Pro-gated (P10-L1).

One flaw stated as unfixable at the edge

The DOM-based XSS (P10-F1) is documented as impossible to block with any WAF and requiring a code-level fix — it is not papered over as if a rule could stop it. The deleted-rule breach is calibrated as an error-based **indicator**, not a clean exploit (P10-L2).

Recovery proven as a loop, not yet a vault

The recovery is proven as a loop with an off-platform, integrity-verified backup — the 3-2-1 principle. It is **not** yet a true immutable/air-gapped vault: the backup lives on the same machine. That stronger guarantee is deferred to the AWS project and named as future work, not claimed here.

TEST 09 / SECURITY TRAINING | LAB EVIDENCE

Privacy and Responsible Use Record

Prepared record · proposed practice

01 — Define the workspace and purpose

Record where the deployment and its evidence live and which tools may assist. Keep the engagement self-contained, note the task and the assets it touches, and define who may access its records.

02 — Keep it synthetic and owned

Attack only a zone, domain and origin the agency owns, with a deliberately-vulnerable target and invented data (`admin@juice-sh.op`). Never point the edge or the attacks at a system that is not yours; a provider's policy permits testing your own assets only.

03 — Protect the keys and the origin

Treat the Cloudflare API token and tunnel credentials as server-only secrets — read them from environment variables, never commit or screenshot them — and keep the origin's address unpublished. The distinction between a public hostname and a private origin is itself a control.

04 — Minimise what is shared

Before sharing evidence, screen screenshots and exports — not just prose — for anything a firm would redact: the operator's own public IP, ISP/ASN and geo, tokens, and any config export that exposes unrelated infrastructure. Mask those; keep payloads, rule names, HTTP codes and synthetic data visible.

05 — Review provider and AI handling

Before sending any project data or output to a hosted service — including an AI assistant — review its retention, training-use and deletion terms. A familiar brand or paid plan is not a data-handling guarantee; no provider-compliance review is claimed here.

06 — Report honestly and keep records

Compare every written claim against the captures: a block is only a block with a 403, an indicator is named as an indicator (not an exploit), and a control an edge cannot provide is said so plainly. Note which tools assisted, and keep the originals preserved separately.

Draws on NIST SP 800-115, the OWASP Top 10, and Cloudflare's Scans and Penetration Testing policy.

PROVENANCE

Evidence notes and references

Keeping the evidence and its interpretation traceable

Evidence notes — provenance and reading limits

Technical screenshots are original and preserved in the per-test folders under descriptive filenames. Where a capture contained the operator's own source IP, ISP/ASN or geolocation, those regions are redacted to a real-engagement standard; attack payloads, rule names, HTTP status codes, request paths, timestamps, synthetic data and infrastructure identifiers are kept visible as the evidence. The scope, architecture and privacy records are prepared report pages. The WAF rule is preserved as a JSON backup and the detector as a Python script in their stage folders; each finding stage also carries a plain-language Test-NN-finding-notes.md record.

References

REF	SOURCE	SCOPE
R1	OWASP Top 10 (2021)	Injection (A03), Security Misconfiguration (A05), Cryptographic Failures (A02) — the classes used to rate the results.
R2	Cloudflare Scans & Penetration Testing policy	Customer permission to test one's own assets; throttle scans; no DoS; never *.cloudflare.com.
R3	NIST SP 800-115	Technical guide to security testing: rules of engagement, scope and reporting.
R4	NIST Cybersecurity Framework 2.0	The Detect and Recover functions — monitoring/alerting and backup/restoration (ransomware resilience).
R5	Tooling	cloudflared (Tunnel), Cloudflare WAF custom rules + GraphQL Analytics API, cf-waf-detector.py (stdlib Python), curl from Kali, SHA-256 + JSON/DNS export for config backup. Free/open-source tools that are the real industry stack; commercial equivalents include Akamai, AWS WAF, Splunk and sqlmap.

References inform the recommendations; they do not certify the lab or grant testing permission. This is a controlled training-lab portfolio, not a client assessment or a legal opinion. Nunito is used from the supplied font package.